

Lecture 2

Mathematical Foundations of Machine Learning

AI for Geometry — MM845

Edward Hirst

Instituto de Matemática, Estatística e Computação Científica
Universidade Estadual de Campinas

`ehirst@unicamp.br`

Agosto 2026

Overview

1. What is Machine Learning?
2. Learning as a Variational Problem
3. Generalisation
4. The Geometry of Data
5. Optimisation & Pipelines
6. Summary

Recap (Lecture 1): The working environment — Python, Git/GitHub, and AI agents.

Guiding question: *What exactly is “learning”, stated as a definition for a mathematician?*

What is Machine Learning?

What is AI?

AI $\implies$ Methods of Computational Statistics

- $\rightarrow$ AI methods rely on an empirical rule: more data $\implies$ better performance.
- $\rightarrow$ Mathematics is rich in data, which is *cheap* to sample at *scale*, and *noiseless*!

The Problem AI approaches

Given *finite samples* from some structured object, recover (an approximation to) the structure. Machine learning is a family of variational methods for doing this computationally at scale.

- Predict the topology of a hypersurface from its defining data $\rightsquigarrow$ **supervised learning**.
- Detect that a point cloud in $\mathbb{R}^{100}$ secretly lies on a 2-manifold $\rightsquigarrow$ **unsupervised learning**.
- Search a space of triangulations for one minimising a quantity $\rightsquigarrow$ **reinforcement learning**.
- Approximate a Ricci-flat metric with no closed form $\rightsquigarrow$ **neural PDEs**.

Thesis of this lecture

Learning = **approximation theory** + **optimisation** + **statistics**.

Each ingredient carries geometric content, mathematics is the foundation for all three.

The Formal Setup

Definition (supervised learning)

Let $(X \times Y, \mu)$ be a measure space of *inputs* and *labels*. Given independent, identically distributed (i.i.d.) samples $\{(x_i, y_i)\}_{i=1}^N \sim \mu$, a *hypothesis space* $\mathcal{H} \subseteq \{f : X \rightarrow Y\}$, and a *loss* $\ell : Y \times Y \rightarrow \mathbb{R}_{\geq 0}$, minimise the **risk**

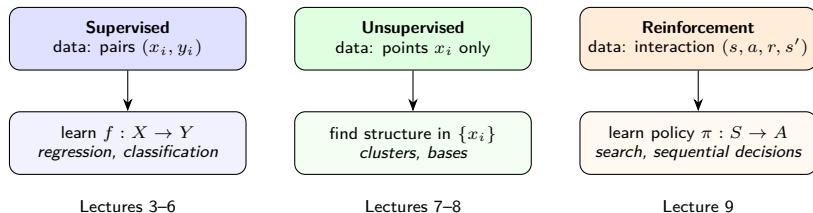
$$\mathcal{R}(f) = \int_{X \times Y} \ell(f(x), y) d\mu(x, y), \quad f \in \mathcal{H}.$$

- μ is **unknown** — we only see samples. This is what separates learning from classical approximation.
- $\mathcal{H}$ is our *ansatz space*: linear maps (Lecture 3), neural networks (Lecture 4), equivariant networks (Lecture 10), ...
- ℓ encodes what “close” means: squared error $\|f(x) - y\|^2$ for regression, cross-entropy for classification, geometric equations for neural PDEs.
- A *regulariser* $\Omega(f)$ may be added — exactly as in Tikhonov regularisation of ill-posed inverse problems.

Geometer's translation

This is a variational problem over a function space, with the measure known only empirically. Everything else in the course is choices of $\mathcal{H}$, ℓ , and the minimisation algorithm.

Three Learning Paradigms



- **Supervised:** $\mapsto$ *Labelled data*.
Approximate a known or unknown map from labelled examples (local curvature from manifold point samples, invariants from defining polynomials).
- **Unsupervised:** $\mapsto$ *Unlabelled data*.
Discover clusters, low-dimensional structure, or identify outliers.
- **Reinforcement:** $\mapsto$ *No data*.
An agent generates its own data by acting and receiving rewards, optimising over *strategies*.

Empirical Risk and Generalisation

Empirical risk minimisation (ERM)

We cannot evaluate $\mathcal{R}(f)$, so we minimise the **empirical risk** on training samples:

$$\widehat{\mathcal{R}}_N(f) = \frac{1}{N} \sum_{i=1}^N \ell(f(x_i), y_i) \xrightarrow{N \rightarrow \infty} \mathcal{R}(f) \quad (\text{law of large numbers, pointwise in } f).$$

- Danger: convergence is *not uniform* over rich $\mathcal{H}$. A flexible f can interpolate the samples ($\widehat{\mathcal{R}}_N = 0$) while $\mathcal{R}(f)$ is terrible: **overfitting**.
- Classical picture: fitting N points with a degree- N polynomial — perfect fit, wild oscillation.
- **Generalisation gap**: $\mathcal{R}(f) - \widehat{\mathcal{R}}_N(f)$, controlled by capacity of $\mathcal{H}$ vs N .

The universal protocol

Split data into:

train (fit parameters) / **validation** (tune choices) / **test** (independent assessment).

The test set is your referee — corrupting it invalidates the experiment.

Capacity, Bias & Variance

Bias–variance decomposition (squared loss)

$$\mathbb{E}_{\mathcal{D}, \varepsilon} \left[\left(f_{\mathcal{D}}(x) - y \right)^2 \right] = \underbrace{\left(\mathbb{E}_{\mathcal{D}}[f_{\mathcal{D}}(x)] - f^*(x) \right)^2}_{\text{Bias}^2: \text{ansatz too rigid}} + \underbrace{\mathbb{E}_{\mathcal{D}} \left[\left(f_{\mathcal{D}}(x) - \mathbb{E}_{\mathcal{D}}[f_{\mathcal{D}}(x)] \right)^2 \right]}_{\text{Variance: ansatz too sensitive}} + \underbrace{\sigma^2}_{\text{irreducible noise}}$$

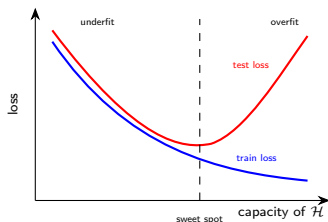
...for $y = f^*(x) + \varepsilon$, $\mathbb{E}(\varepsilon) = 0$, $\text{Var}(\varepsilon) = \sigma^2$.

- Small $\mathcal{H} \implies$ high bias
Architecture too simple to represent the truth.
- Large $\mathcal{H} \implies$ high variance
Architecture too expressive & overfits to noise.

To alleviate these competing effects:

- Change architecture: expressivity, or symmetry (Lecture 10).
- Regularisation: Overparameterise but punish complexity in training.
- More data: Effects of noise reduced.

...note in maths data: $\varepsilon = 0$!!



The *classical* U-curve.

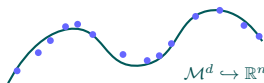
Modern twist: heavily overparameterised networks can descend again!
— “double descent” (Lecture 4).

The Manifold Hypothesis

Manifold hypothesis

Real datasets in $\mathbb{R}^n$ typically concentrate near a submanifold (or stratified space) $\mathcal{M}$ of dimension $d \ll n$. Learning succeeds when methods exploit this *intrinsic* low dimension.

- Images of a rotating object: pixels live in $\mathbb{R}^{10^6}$, but the data is a curve (or $SO(3)$ -orbit) — intrinsic dimension ≤ 3 .
- Manifold data we represent in ambient spaces literally does match this common theme in AI.
- Geometric priors (symmetry, smoothness, locality) restrict $\mathcal{H}$ to functions adapted to $\mathcal{M}$: already lowering degrees of freedom and thus effective dimension.



Samples (blue) concentrating near a low-dimensional submanifold.

In Tutorial 2

Sample from *known* manifolds and varieties, so you can compare what ML recovers against ground truth — a luxury real-world data never offers.

High Dimensions: Curse and Blessing

Curse of dimensionality

Covering $[0, 1]^n$ at resolution ε needs ε^{-n} points: sample complexity of *generic* approximation is exponential in n . Naive grid / nearest-neighbour methods become infeasible quickly.

Concentration of measure

On S^{n-1} , a Lipschitz function ($|f(x) - f(y)| \leq |x - y|$) concentrates near its median m_f :

$$\mathbb{P}(|f - m_f| > t) \leq 2e^{-c n t^2}.$$

- The inner product measure $x \mapsto \langle x, u \rangle$ is Lipschitz with median 0, therefore high $n \implies$ random vectors are nearly orthogonal.
- For distances $|X - Y|^2 = |X|^2 + |Y|^2 - 2\langle X, Y \rangle$, so on the sphere distances between random points concentrate; all points 'look' equidistant.
- Why ML works in high-dimensions: real data has low *intrinsic* dimension (manifold hypothesis), and good hypothesis spaces (convolutions, attention, equivariant maps) exploiting *structure*, not raw dimension.
- The blessing of higher-dimensional concentration is it applies along the number of samples N also, such that concentration leads to stability of stochastic optimisation.

Optimisation: Gradient Flow and Its Discretisation

Stochastic Gradient Descent (SGD)

Parameterise $\mathcal{H} = \{f_\theta : \theta \in \mathbb{R}^p\}$ and set $L(\theta) = \widehat{\mathcal{R}}_N(f_\theta)$. Training is the discretised negative gradient flow

$$\dot{\theta} = -\nabla L(\theta) \quad \rightsquigarrow \quad \theta_{t+1} = \theta_t - \eta \nabla L(\theta_t),$$

with *learning rate* η the step size, and $L(\theta_t)$ computed over *batches* of data.

- The random process of sampling batches of data for each step is what makes the learning process *stochastic*!
- L may be convex (Lecture 3) $\Rightarrow$ global convergence guarantees; or non-convex (Lecture 4) $\Rightarrow$ landscape has saddles, valleys, symmetries; geometry of the landscape governs trainability $\mapsto$ whole field of geometry for AI!
- The metric matters: gradient depends on the inner product on *parameter* space of θ — many other optimisers are changes of this metric.

More Advanced Optimisers

- Momentum averages gradients over time: $m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla L(\theta_t)$,
...stabilising and accelerating consistent descent directions.
- RMSProp rescales updates using a moving average of $(\nabla L(\theta_t))^2$, Adam combines both:

$$\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t + \varepsilon}}, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla L(\theta_t))^2.$$

The Practical Pipeline: Data Management

The pipeline for AI projects

Generate data → Representation → Preprocessing → Split → Train → Evaluate → Interpret

...mathematical intuition is useful at all stages!

- **Representation:** how do you encode a variety, triangulation, metric, etc, efficiently as a tensor? The single most consequential design choice.
- **Normalisation:** centre/scale features with the objective of positioning the loss landscape to be optimal for the training process.
- **Splits:** random splits can leak information when data has symmetry — e.g. the same polytope in two orientations landing in train *and* test. Split by *orbit*, not by sample.
- **Measures:** accuracy, MSE, R^2 — always compare against a trivial baseline (predict the mean; predict the majority class).
- **Logging:** record seed, config, commit hash for every run (Lecture 1 practices) $\mapsto$ reproducibility.

Caution at Interpretation

Be suspicious of good results, check for bugs intermittently and test trained models extensively.
...that being said, successes have been shown to be surprising in the past!

Summary

Key points

- Learning is a variational problem:

Iteratively minimise expected loss over a hypothesis space,
...knowing the measure only through samples.

- Three paradigms — *supervised*, *unsupervised*, *reinforcement* — differ in data structure and objective, not in spirit.
- Seek models which generalise beyond shown, this requires tuning capacity via: bias & variance, regularisation, and correct structural priors.
- Data concentrates on low-dimensional geometric objects; exploiting this is often what makes high-dimensional learning successful.
- Training is *discretised gradient flow*; the loss landscape is a geometric object we will keep returning to.

Next

Tutorial 2: Function approximation on sampled manifolds as energy minimisation.

Lecture 3: Linear models — the convex world where the theory is complete.